

What This Project Is:

A Python app that builds a dynamic daily timetable for a student from their fixed timetable, co-curricular schedule, tasks (deadline, difficulty, priority), and sleep needs. It protects sleep and deadlines; everything else is negotiable. Built with Pydantic, OR-Tools CP-SAT, SQLite, and pytest.

Locked-In Design Decisions:

- Internal unit: 15-minute slots on one continuous multi-day timeline. Tasks and fixed blocks can cross midnight.
- Breaks are buffer time (`buffer_slots`) after items, not scheduled events. Mandatory after every task-task, task-block, and block-task pair; NOT enforced between two fixed blocks (they may be back-to-back).
- Tasks can be split into sessions (`max_session_slots` per task), each with a mandatory gap between sessions.
- Study sessions default to a 2h session cap and prefer spreading across different days (`same_day_penalty`), falling back to doubling up only when the window is tight.
- SleepRule per night: target/min length, bedtime window, skip flag. Missing minimum sleep or skipping a night is a hard warning; missing a task deadline is a hard warning; other drops are soft.
- ProfileSettings holds every tunable constant (`buffer_slots`, all penalty weights, sleep/session defaults) per student — nothing about “how the algorithm behaves” is hardcoded any more.
- Objective priority (highest cost first): sleep minimum > sleep target > task fit/priority > same-day spread > bedtime drift.

ASG Reamaining Sctions:

Remaining for 3.2, in priority order

1. **The review-before-save flow — the biggest real gap.** Right now `extract_schedule()` just returns data; nothing lets you look at what got extracted, accept or reject individual items, or fix a wrong placeholder duration/priority before it's saved. This needs the same pattern as `reflect.py`: show each item, confirm, then call `db.py`'s `add_fixed_block` / `add_task` / `add_exam`. Without this, extraction is a demo, not a usable feature.
2. **A real plan anchor.** `expand_weekly_pattern` needs `plan_start_date` and `num_days` — nothing currently lets a student say "my semester starts Aug 31, runs 16 weeks." This has to exist before extraction output can become real `FixedBlock`s in the database.
3. **Task placeholder review.** `ExtractedTask`'s `duration_slots` / `priority` / `difficulty` are guesses (1h, medium, medium) — the review step needs to let these be edited, not just accepted as-is.
4. **Deduplication on re-upload.** If someone uploads a corrected image or the same document twice, nothing stops duplicate entries. `db.py` already has `clear_fixed_blocks` for exactly this — it just isn't wired into an extraction flow yet.
5. **PDF path is untested.** Every real test so far has been images. The Anthropic PDF branch in `llm_backends.py` has never actually run against a real PDF.
6. **Cost gating.** Per the monetization plan, extraction is an LLM cost center like reflections — needs to join that same gate once built, not ship unlimited/free.
7. **Broader format testing** — parked deliberately for the model comparison, not urgent now, but worth remembering: three real timetables tested (two clean, one genuinely hard) isn't a large sample.

Items 1-3 are the real blockers to this being usable at all; 4-7 matter before it's production-ready.

3. V2 — Core Product (*remaining, reordered for real dependencies*)

3.1. **Persistence for schedule inputs** (*new prerequisite*) — `db.py` currently saves settings and reflections, but not a student's actual classes/tasks/exams. This needs its own tables before extraction has anywhere real to save into.

3.2. **Import a schedule from an image or PDF** — **flagged high priority, as you asked.** Built on Claude's native image/PDF input (a vision call, structured via the same tool-use pattern as `reflection.py`) to extract classes/tasks into real `FixedBlock` / `DynamicTask` objects, which the student then reviews before saving — same review-before-commit pattern as the reflection loop. Important cost note: like the reflection loop, this is a real LLM API call, not free local compute — it belongs behind the same monetization gate as reflections, not bundled with the free scheduler.

3.3. **Task-dropping decisions when everything can't fit** — manual (student picks), semi-automatic (app proposes by priority/deadline, student confirms), or automatic (app just drops by rule). A per-student setting, and a natural thing for the reflection loop to eventually adjust.

3.4. **Daily commute** — either representable today as a plain `FixedBlock`, or worth a dedicated concept that auto-generates around the first/last class each day so it's entered once, not daily.

3.5. Three-tier preference system (locked / user / model-learned) — still just a design note.

3.6. Upper bounds on adjustable settings — nothing currently stops a penalty weight drifting indefinitely.

3.7. Monetization gating in code — scheduler free, reflection *and* image-import gated (both are real LLM cost centers).

3.8. Frontend — schedule view, input flow, reflection/proposal screens, image-upload flow, settings view.

4. V3 — Learning From History

4.1. Learn from accumulated reflection history, not just the latest one.

4.2. Cognitive-flow modeling — from your very first one-line pitch ("protecting sleep and promoting cognitive flow"), never actually built: sequencing that avoids stacking hard tasks back-to-back even when slots are technically free.

5. Models — What to Test, and How

Same as before: ~10-15 real reflections through Haiku / Llama-via-Groq (free) / Sonnet, compared on a real rubric before committing spend. Once image extraction exists, the same evaluation applies there too — vision-model accuracy on messy real screenshots matters as much as text-proposal quality does.

6. Other Ideas (*honestly labeled: differentiator vs. parity*)

- Calendar export (`.ics`) / live Google Calendar sync — **parity**, most competitors have this.
- LMS/portal auto-import — **parity**, this is DormWay's whole core feature; low priority to chase directly.
- Notifications/reminders — **parity**.
- Semester analytics (hours per subject, missed-deadline trend, sleep-debt trend) — **differentiator**, since it's built on data none of these competitors actually have (real time-block history + sleep protection data).
- Accountability/social features — **parity-ish**, several planners have light versions of this.
- Offline-first / multi-device — **parity**, expected baseline for a real mobile app eventually.
- Grade-weighted priority (*new idea, worth a flag*) — DormWay tracks grading weight from syllabi; adopting a similar `grade_weight` field on tasks would be useful, but leaning into it too far pulls your app toward *being* a grade tracker, which is squarely DormWay's territory. Worth having as a minor field, not a headline feature.